



Saint Columban College
College of Computing Studies
Pagadian City



Flag in Flame

Flag in Flame 



Easy Forensics picoMini by CMU-Africa browser_webshell_solvable

AUTHOR: PRINCE NIYONSHUTI N.

Description

The SOC team discovered a suspiciously large log file after a recent breach. When they opened it, they found an enormous block of encoded text instead of typical logs. Could there be something hidden within? Your mission is to inspect the resulting file and reveal the real purpose of it. The team is relying on your skills to uncover any concealed information within this unusual log. Download the encoded data here: [Logs Data](#). Be prepared—the file is large, and examining it thoroughly is crucial .

Hints

1

Use `base64` to decode the data and generate the image file.

18,328 users solved



83% Liked



picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Flag in Flame

Category: Forensics

Date: 03/10/26



I. Objective

The objective of this challenge is to analyze a provided file and identify hidden data within it using forensic techniques and decoding methods in order to retrieve the hidden flag.

II. Background

In digital forensics, files can contain **encoded or disguised data** that must be decoded to reveal their true content. One common encoding method is **Base64**, which converts binary data into readable ASCII characters so it can be easily transmitted or stored in text files.

Sometimes encoded files hide another file type entirely. For example, a text file may actually contain **Base64-encoded image data**. Once decoded, the output may reveal an image or another file format.

Another encoding scheme frequently used in cybersecurity challenges is **hexadecimal (hex) encoding**, which represents data using base-16 numbers. By converting hex back to readable text, hidden information such as flags can be revealed.

This challenge demonstrates how encoded data may hide multiple layers of information that must be decoded step by step.

III. Tool Used

- **Kali Linux (VirtualBox)** - Used as the environment for performing the analysis.
- **Linux Terminal Commands (cat, base64)** - Used to inspect and decode file contents.
- **CyberChef** - Used to decode hexadecimal data.
- **Online Image Text Reader / OCR Tool** - Used to extract text from the image.

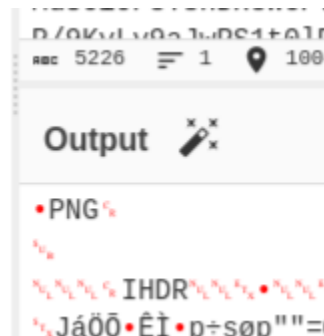
IV. Methodology



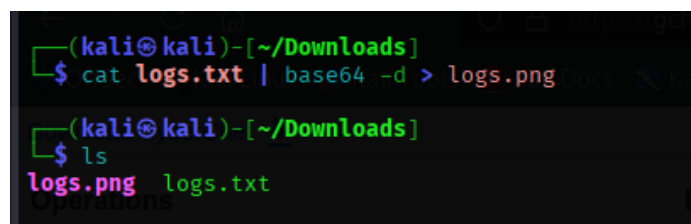
- ## 5. Decoded the Base64 content

[illegible]

6. Noticed it has “PNG” as output.



7. Rea the content then decoded the base64 as a PNG image using the command:



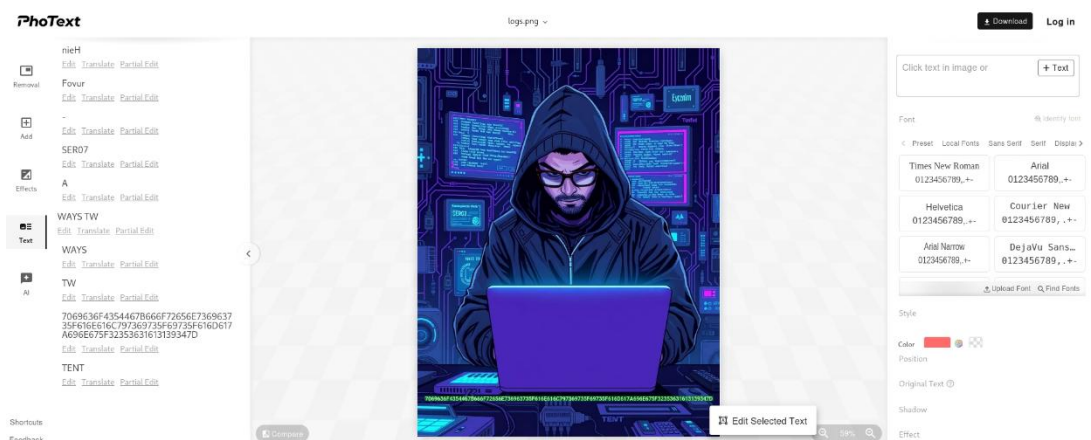


cat logs.txt | base64 -d > logs.png

8. Opened the newly created **logs.png** file and observed that it contained a **hexadecimal string displayed in the image**.

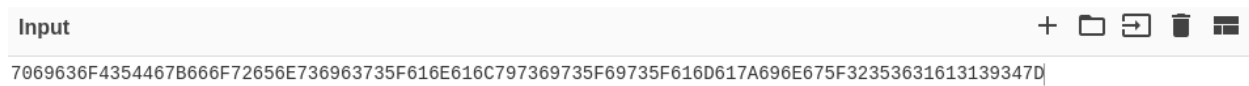


9. Used an **online image text extraction tool (OCR)** to copy the hex string from the image.

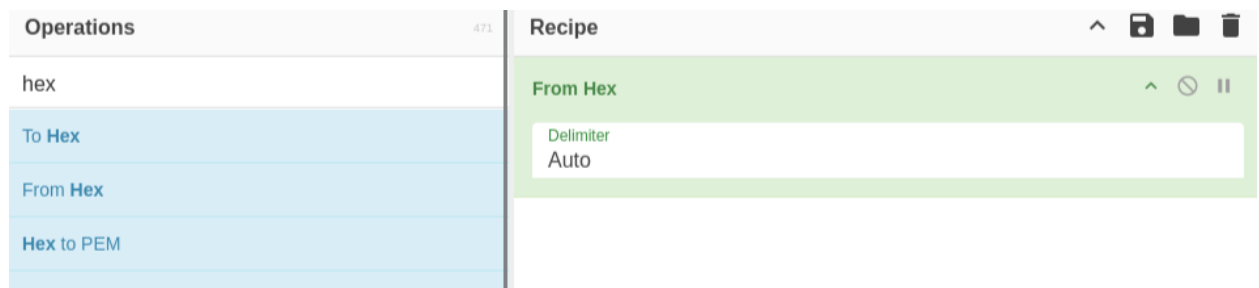




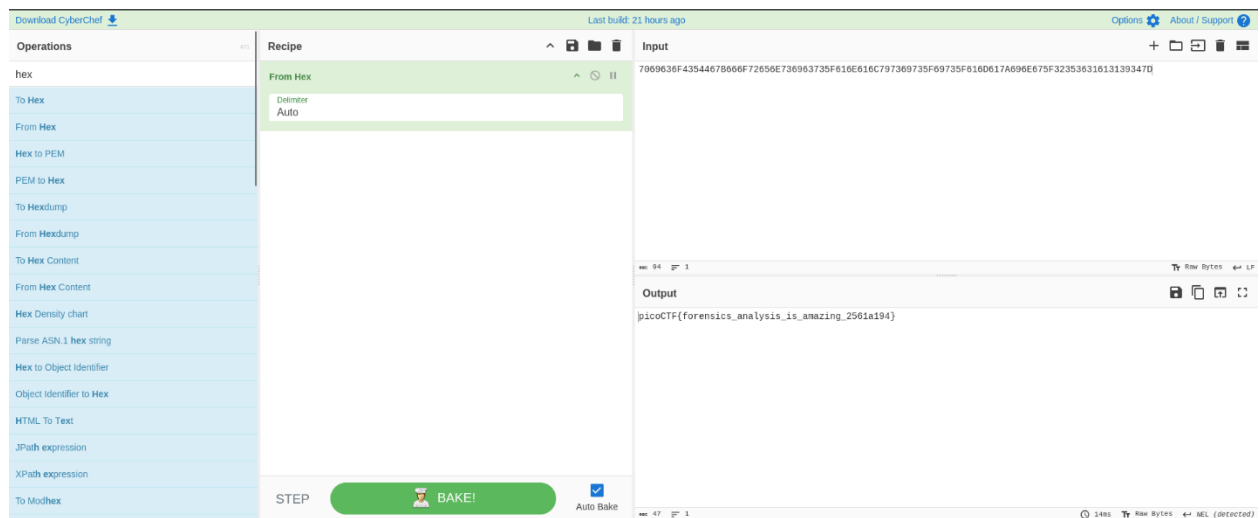
10. Pasted the extracted hex string into **CyberChef**.



11. Applied the **From Hex** operation to decode the hexadecimal data.



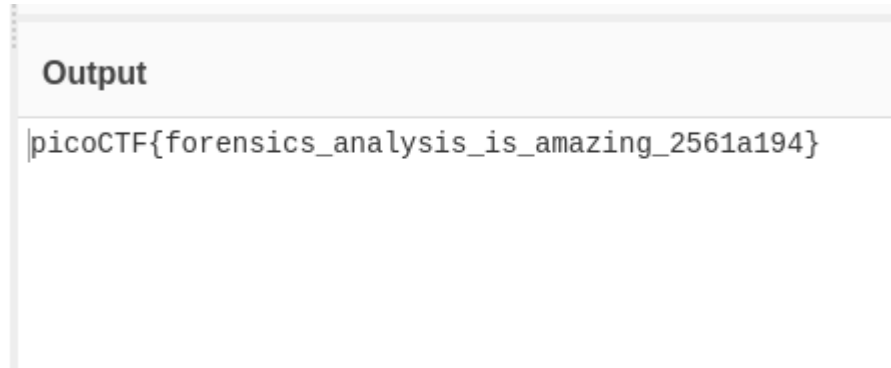
12. The decoded output revealed the challenge flag.





V. Result

picoCTF{forensics_analysis_is_amazing_2561a194}



VI. Explanation

Explanation of the command:

- **cat logs.txt** → reads the contents of the file
- **|** → pipes the output to another command
- **base64 -d** → decodes the Base64 encoded data
- **> logs.png** → saves the decoded output as an image file

The challenge required analyzing encoded data hidden inside a text file. Initially, the file appeared to be a normal text file, but inspecting its contents revealed **Base64-encoded data**.

After decoding the Base64 data, the output produced a **PNG image file**. This indicated that the text file was actually storing an image in encoded form.

When the image was opened, it displayed a **hexadecimal string embedded in the image itself**. Since hexadecimal is another encoding format used to represent binary data, decoding it was necessary to reveal the final message.



By extracting the hex string from the image and decoding it using **CyberChef**, the encoded message was converted back into readable text, which revealed the hidden flag.

This challenge demonstrates the importance of **layered forensic analysis**, where multiple encoding techniques may be used together to conceal information.

VII. Conclusion

The Flag in Flame challenge illustrates how encoded data can hide multiple layers of information within files. By identifying Base64 encoding, converting it into an image file, and then decoding a hexadecimal string embedded in the image, the hidden flag was successfully recovered. This challenge reinforces the importance of understanding encoding techniques and using appropriate forensic tools to uncover hidden data.

— **The Analyst: Hyposelenia**